Dealing With Contension

什麼是 Contention？

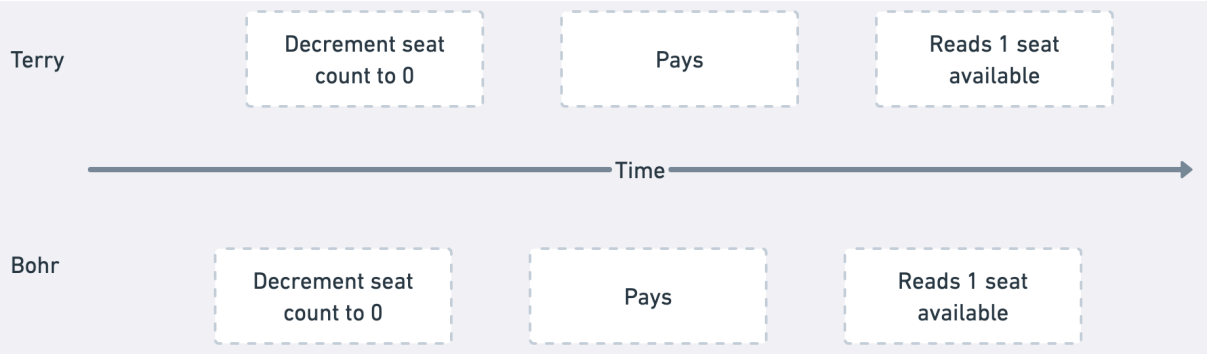
Contention（競爭）發生在多個 process 同時爭奪同一個資源的時候。這可能是搶購最後一張演唱會門票、在拍賣中出價競標，或任何類似的情境。如果沒有適當的處理，就會出現 race condition（競態條件）、重複預訂、以及不一致的狀態。這篇講義會帶你從簡單的 database transaction 走到更複雜的分散式協調機制，說明什麼時候 optimistic concurrency 比 pessimistic locking 更合適，以及如何突破單一節點的限制往外擴展。

問題在哪裡

想像一下線上購買演唱會門票。The Weeknd 演唱會只剩 1 個座位，Terry 和 Bohr 都想要這個最後的座位，在完全同一個瞬間點下「立即購買」。如果沒有適當的協調，以下的事情就會發生：

1. Terry 的請求讀取：「剩 1 個座位」
2. Bohr 的請求讀取：「剩 1 個座位」（兩個讀取都在任何一個寫入發生之前完成）
3. Terry 的請求判斷 1 ≥ 1（有座位），進入付款流程
4. Bohr 的請求判斷 1 ≥ 1（有座位），進入付款流程
5. Terry 被扣了 $500，座位數減為 0
6. Bohr 被扣了 $500，座位數減為 -1
7. Terry 和 Bohr 都收到確認信，而且是完全一樣的座位號碼

兩個人都到了現場，都以為 Row 5, Seat 12 是自己的。其中一個人要被請出去，場館還要處理退款和兩個怒氣沖沖的顧客。



buildmoat.org

Race condition 之所以發生，是因為 Terry 和 Bohr 都讀到了同樣的初始狀態（1 個座位），而在任何一方的更新生效之前，雙方都完成了讀取。等到 Bohr 的更新執行時，Terry 已經把座位數降到 0 了，但 Bohr 的邏輯是基於那個過時的「剩 1 個座位」。

這個 race condition 的根本原因是讀取和寫入不是原子的（atomic）。在「讀取當前狀態」和「根據那個狀態做更新」之間，存在一個時間差，就是在那個微小的窗口裡（在記憶體中是微秒，透過網路則是毫秒），一切都可能改變。

當你進行擴展，問題只會更嚴重。10,000 個並發用戶同時打同一個資源，就算再小的 race condition 窗口也會製造大量衝突。隨著規模繼續成長，你可能還需要跨多個節點協調，那複雜度又是另一個層次了。

要把這件事做對，我們需要某種形式的同步機制。

解法的架構

處理 contention 問題的解法有一個很自然的複雜度遞進：我們從使用 atomicity 和 transaction 的單一 database 解法開始，在並發存取造成衝突時加入協調機制，最後在需要跨多個 database 時才進入分散式協調。

單節點解法（Single Node Solutions）

當你所有的資料都在單一 database 時，contention 的解法相對直觀，但還是有一些重要的陷阱要注意。

Atomicity（原子性）

在拿出複雜的協調機制之前，atomicity 就能解決很多 contention 問題。Atomicity 的意思是，一組操作全部成功，或者全部失敗，沒有部分完成的狀態。如果你要在兩個帳戶之間轉帳，要麼扣款和入帳都發生，要麼兩件事都不發生。

Transaction 是 database 提供 atomicity 的方式。一個 transaction 是一組被當作單一單元處理的 database 操作。你用 `BEGIN TRANSACTION` 開始，執行你的操作，然後用 `COMMIT`（儲存變更）或 `ROLLBACK`（全部取消）結束。

```
BEGIN TRANSACTION;

-- 從 Terry 的帳戶扣款
UPDATE accounts SET balance = balance - 100 WHERE user_id =
'Terry';

-- 存入 Bohr 的帳戶
UPDATE accounts SET balance = balance + 100 WHERE user_id =
```

buildmoat.org

```
'Bohr';

COMMIT; -- 兩個操作一起成功
```

如果在這個 transaction 過程中任何事情出錯，比如 Terry 餘額不足、Bohr 的帳戶不存在、或 database 崩潰，整個 transaction 就會被 rollback。這防止了金錢憑空消失或憑空出現。

這些範例使用 SQL，因為 relational database 以強大的 ACID 保證聞名（ACID 裡的「A」就代表 Atomicity）。但很多 database 都支援 transaction，包括 NoSQL database 如 MongoDB（multi-document transaction）和 DynamoDB（transaction 操作），以及分散式 SQL database 如 CockroachDB。這些概念不管用什麼 database 都適用。

對演唱會門票購買的情境，atomicity 確保多個相關操作一起發生。購票不只是把座位數減一，你還需要建立一筆票券記錄：

```
BEGIN TRANSACTION;

-- 確認並保留座位
UPDATE concerts
SET available_seats = available_seats - 1
WHERE concert_id = 'weeknd_tour';

-- 建立票券記錄
INSERT INTO tickets (user_id, concert_id, seat_number, purc
hase_time)
VALUES ('user123', 'weeknd_tour', 'A15', NOW());

COMMIT;
```

如果任何一個操作失敗，整個 transaction 就 rollback。你不會碰到座位已扣但票券沒建立的情況。

但即使有了這個原子 transaction，還有一個 atomicity 本身無法解決的微妙問題：兩個人還是可能同時預訂到同一個座位。原因如下：Terry 和 Bohr 可以同時開始各自的 transaction，兩個都讀到 `available_seats >= 1`（都看到剩 1 個座位），然後兩個都執行各自的 `UPDATE`。每個 transaction 本身是原子的，所以兩個都成功了，但現在我們把 1 個座位賣出了 2 張票。

buildnoat.org

問題在於，transaction 只保證自身內部的 atomicity，但無法阻止其他 transaction 同時讀取一樣的資料。我們需要協調機制來解決這個問題。

Pessimistic Locking（悲觀鎖）

Pessimistic locking 透過事先取得 lock 來防止衝突。之所以叫「悲觀」，是因為它對衝突持悲觀態度，假設衝突一定會發生，所以先預防它。

我們可以用明確的 row lock 來修正 race condition：

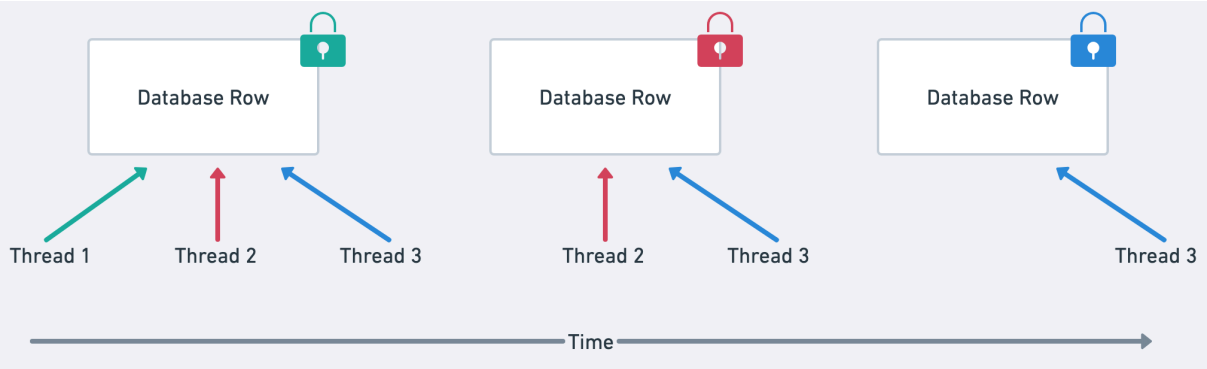
> 實際的票務系統當然會有票券預留（reservation）的概念來提升用戶體驗，但為了讓範例保持簡單，我們先這樣處理，預留機制的細節會在後面的章節討論。

```
BEGIN TRANSACTION;

-- 先鎖住這一行，防止 race condition
SELECT available_seats FROM concerts
WHERE concert_id = 'weeknd_tour'
FOR UPDATE;

-- 現在可以安全地更新座位數
UPDATE concerts
SET available_seats = available_seats - 1
WHERE concert_id = 'weeknd_tour';

-- 建立票券記錄
INSERT INTO tickets (user_id, concert_id, seat_number, purc
hase_time)
VALUES ('user123', 'weeknd_tour', 'A15', NOW());

COMMIT;
```

buildmoat.org

`FOR UPDATE` 子句在讀取之前就對 concert 那一行取得一個排他鎖（exclusive lock）。當 Terry 執行這段程式碼時，Bohr 完全相同的 transaction 會在 `SELECT` 那一行卡住，直到 Terry 的 transaction 完成。這防止了兩個人看到同樣的初始座位數，確保同一時間只有一個人能進行「讀取並更新」的操作。

Lock 在這個脈絡下是一個阻止其他 database connection 存取同一筆資料的機制，直到 lock 被釋放。PostgreSQL 和 MySQL 這類 database 可以同時處理幾千個並發連接，但 lock 確保了同一時間只有一個連接可以修改特定的 row（或一組 row）。



效能考量非常重要。 你希望鎖住盡可能少的 row、維持盡可能短的時間。鎖住整張表會殺死並發能力；持有 lock 幾秒而不是幾毫秒就會製造瓶頸。在我們的例子裡，我們只在購票過程中短暫鎖住一個特定的 concert row。

Isolation Level（隔離等級）

除了用 `FOR UPDATE` 明確鎖 row，你也可以讓 database 透過調高 isolation level 來自動處理衝突。Isolation level 控制並發的 transaction 之間能看到多少彼此的變更，也就是每個 transaction 對其他 transaction 的工作有多「隔離」。

大多數 database 支援四個標準 isolation level（這些是不同的選項，不是進階程度）：

- **READ UNCOMMITTED**：可以看到其他 transaction 尚未 commit 的變更（極少使用）
- **READ COMMITTED**：只能看到已 commit 的變更（PostgreSQL 預設）
- **REPEATABLE READ**：同一個 transaction 內多次讀取同一資料，結果保持一致（MySQL 預設）
- **SERIALIZABLE**：最強的隔離，transaction 看起來像是一個接一個執行的

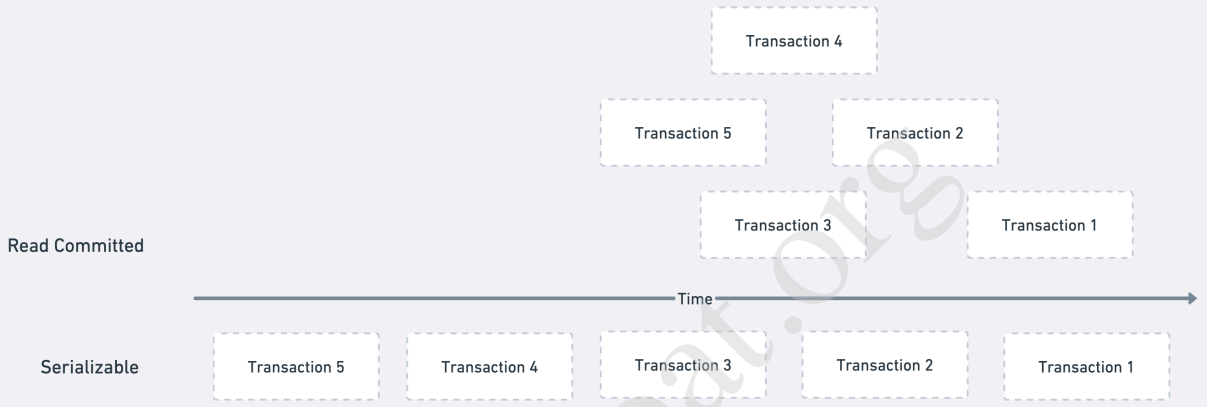
預設的 READ COMMITTED 或 REPEATABLE READ 都還是允許我們的演唱會門票 race condition，因為 Terry 和 Bohr 可以同時讀到「剩 1 個座位」再各自更新。SERIALIZABLE isolation level 透過讓 transaction 看起來逐一執行來解決這個問題：

```
-- 為這個 transaction 設定 isolation level
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;

UPDATE concerts
SET available_seats = available_seats - 1
WHERE concert_id = 'weeknd_tour';
```

buildnoat.org

```
-- 建立票券記錄
INSERT INTO tickets (user_id, concert_id, seat_number, purc
hase_time)
VALUES ('user123', 'weeknd_tour', 'A15', NOW());

COMMIT;
```

在 SERIALIZABLE 模式下，database 會自動偵測衝突，如果兩個 transaction 可能互相干擾，就自動中止其中一個。被中止的 transaction 必須重試。



取捨是：SERIALIZABLE isolation 比明確的 lock 昂貴得多。它需要 database 追蹤所有讀取和寫入來偵測潛在衝突，而且 transaction abort 會浪費必須重做的工作。明確的 lock 讓你精確控制哪些東西要在何時被鎖住，對那些你清楚知道哪些資源需要協調的情境來說，效率更高。

Optimistic Concurrency Control（樂觀並發控制）

Pessimistic locking 假設衝突一定會發生，所以先預防。Optimistic concurrency control（OCC）採取相反的態度，假設衝突很罕見，在衝突發生後才偵測它。

效能上的好處很顯著。不是讓 transaction 阻塞著等待 lock，而是讓它們全部繼續執行，只有真的衝突的才需要重試。在低 contention 的情況下，這完全消除了 locking 的 overhead。

做法很簡單：在你的資料裡加一個版本號。每次更新一筆記錄，就遞增版本號。更新時，同時指定新的值和期望的當前版本號：

```
-- Terry 讀取：concert 剩 1 個座位，版本號 42
-- Bohr 讀取：concert 剩 1 個座位，版本號 42

-- Terry 先嘗試更新：
```

buildmoat.org

```
BEGIN TRANSACTION;
UPDATE concerts
SET available_seats = available_seats - 1, version = versio
n + 1
WHERE concert_id = 'weeknd_tour'
  AND version = 42;  -- 期望的版本號

INSERT INTO tickets (user_id, concert_id, seat_number, purc
hase_time)
VALUES ('Terry', 'weeknd_tour', 'A15', NOW());
COMMIT;

-- Terry 的更新成功，seats = 0，version = 43

-- Bohr 嘗試更新：
BEGIN TRANSACTION;
UPDATE concerts
SET available_seats = available_seats - 1, version = versio
n + 1
WHERE concert_id = 'weeknd_tour'
  AND version = 42;  -- 版本號已過期！

-- Bohr 的更新影響了 0 筆資料——偵測到衝突，transaction rollback
```

Bohr 的更新失敗後，他知道有人修改了這筆記錄。他可以重新讀取當前狀態，確認是否還有座位，然後用新的版本號重試。如果座位已售完，他就會收到清楚的「已售完」訊息，而不是一個莫名其妙的失敗。

重要的是，「版本號」不一定要是獨立的欄位。你可以用資料本身已有的、每次更新都會改變的值。在我們的演唱會範例中，available seats 數量本身就可以當版本：

```
-- Terry 讀取：剩 1 個座位
-- Bohr 讀取：剩 1 個座位

-- Terry 先嘗試更新：
BEGIN TRANSACTION;
UPDATE concerts
SET available_seats = available_seats - 1
WHERE concert_id = 'weeknd_tour'
```

buildmoat.org

```
  AND available_seats = 1;  -- 期望的當前值

INSERT INTO tickets (user_id, concert_id, seat_number, purc
hase_time)
VALUES ('Terry', 'weeknd_tour', 'A15', NOW());
COMMIT;

-- Terry 的更新成功，seats 現在 = 0

-- Bohr 嘗試更新：
BEGIN TRANSACTION;
UPDATE concerts
SET available_seats = available_seats - 1
WHERE concert_id = 'weeknd_tour'
  AND available_seats = 1;  -- 這個值已過期！

-- Bohr 的更新影響了 0 筆資料——偵測到衝突，transaction rollback
```

這個做法之所以有效，是因為我們在確認當前座位數和我們讀到的一樣。如果有人先買走了，座位數就變了，我們的更新就會失敗。

同樣的模式可以應用在其他情境。eBay 競標用當前最高出價作為版本；銀行轉帳用帳戶餘額；庫存系統用庫存數量。任何在記錄被更新時會改變的值，都可以作為你的 optimistic concurrency control 機制。

要注意避免所謂的 **ABA 問題**：thread A 讀到一個值 A，thread B 把它改成 B 再改回 A，然後 thread A 在做 compare-and-swap 時以為什麼都沒變。這在 OCC 中使用簡單版本號而版本號可能繞回來、或記憶體被重複使用時，就可能發生。詳細討論留到後面的 deep dive 章節。

這個方式在衝突不常發生的情況下最合適。對大多數電商情境而言，兩個人在完全同一個瞬間買同一件商品的機率很低。偶爾的重試，比起 pessimistic locking 的持續 overhead 要划算得多。

多節點解法（Multiple Nodes）

我們前面介紹的所有做法都適用於單一 database。但如果需要跨多個 database 協調更新，事情就會複雜很多。

強烈建議：如果你的系統在高 contention 情境下需要強一致性保證，盡一切可能把相關資料放在同一個 database 裡。 十次有九次這是完全可行的，而且能避免分散式協

wulianwat.org

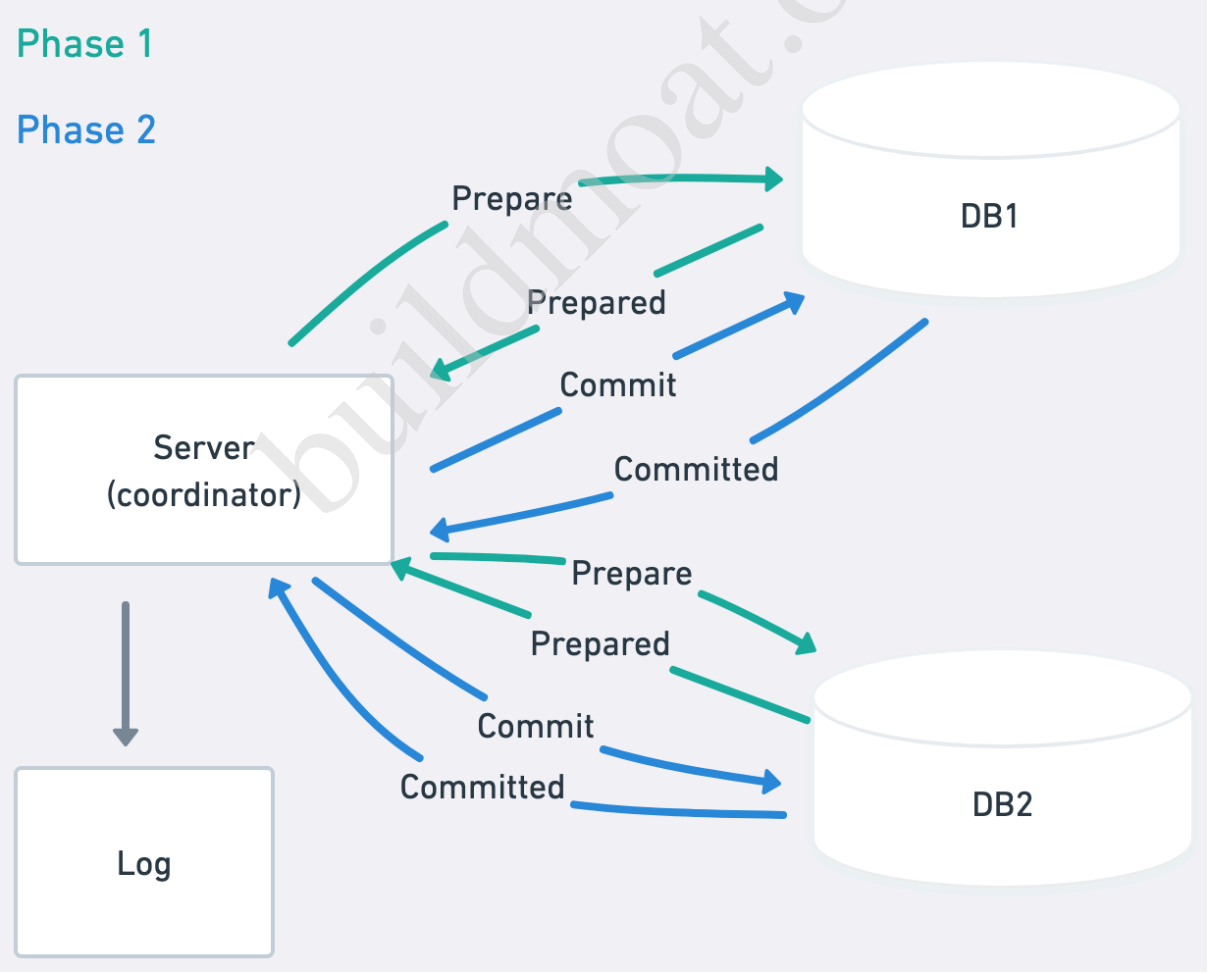
調。，

想像一個 Terry 和 Bohr 帳戶在不同 database 的銀行轉帳。也許你的銀行規模大到需要把用戶帳戶 shard 到多個 database 上。Terry 的帳戶在 Database A，Bohr 的帳戶在 Database B。現在你無法用單一 database transaction 處理這筆轉帳了。Database A 需要從 Terry 的帳戶扣 $100，Database B 需要把 $100 存入 Bohr 的帳戶。兩個操作必須同時成功或同時失敗。如果 Database A 扣了 Terry 但 Database B 存入 Bohr 失敗，錢就從系統裡消失了。

分散式協調有幾個選項，各自有不同的取捨。

Two-Phase Commit（2PC，兩階段提交）

經典的解法是 two-phase commit，你的 transfer service 作為 coordinator，管理跨多個 database participant 的 transaction。Coordinator（你的 service）在第一個階段詢問所有 participant 是否準備好提交 transaction，然後根據所有人是否都準備好了，在第二個階段通知它們 commit 或 abort。



buildnoat.org

關鍵點是：coordinator 在發送任何 commit 或 abort 決定之前，必須先寫入一個持久化的 log。這個 log 記錄了哪些 participant 參與其中，以及 transaction 的當前狀態。如果沒有這個 log，coordinator 崩潰就會製造無法恢復的情況，participant 不知道是該 commit 還是 abort 它們已準備好的 transaction。

跨網路呼叫持有開啟中的 transaction 是非常危險的。 那些開啟中的 transaction 鎖住了 Terry 和 Bohr 的帳戶 row，阻擋任何其他操作存取這些帳戶。如果你的 coordinator service 崩潰，這些 transaction 就永遠開著，可能把帳戶鎖住永遠。生產系統會加上 timeout，在 30 到 60 秒後自動 rollback 已準備好的 transaction，但這又製造了別的問題，合法的慢速操作可能被 rollback，導致轉帳失敗，即使它本來應該成功。

在 prepare 階段，每個 database 做除了最終 commit 之外的所有工作。Database A 開啟一個 transaction，確認 Terry 有足夠的餘額，hold 住 $100，但還不 commit 造成變更已完成，結果尚未永久生效，而其他 transaction 看不到這些變更。Database B 開啟一個 transaction，確認 Bohr 的帳戶存在，準備好加入 $100，但也還不 commit。用 SQL 來表示大概長這樣：

```
-- Database A 在 prepare 階段
BEGIN TRANSACTION;
SELECT balance FROM accounts WHERE user_id = 'Terry' FOR UP
DATE;
-- 確認餘額 >= 100
UPDATE accounts SET balance = balance - 100 WHERE user_id =
'Terry';
-- Transaction 持續開啟，等待 coordinator 的決定

-- Database B 在 prepare 階段
BEGIN TRANSACTION;
SELECT * FROM accounts WHERE user_id = 'Bohr' FOR UPDATE;
-- 確認帳戶存在且有效
UPDATE accounts SET balance = balance + 100 WHERE user_id =
'Bohr';
-- Transaction 持續開啟，等待 coordinator 的決定
```

如果兩個 database 都能成功準備，你的 service 就通知它們 commit 各自開啟中的 transaction。如果任何一方失敗，兩者都 rollback。

Two-phase commit 保證了跨多個系統的 atomicity，但它昂貴而且脆弱。如果你的 service 在 prepare 和 commit 之間崩潰，兩個 database 就會陷入有開啟中

buildmoat.org

transaction 的不確定狀態。如果任何一個 database 變慢或無法存取，整個轉帳就卡住了。網路分割（network partition）可能讓系統進入不一致的狀態。

Distributed Lock（分散式鎖）

對較簡單的協調需求，可以使用 distributed lock。與其協調複雜的 transaction，只需確保在整個系統中，同一時間只有一個 process 可以操作特定資源。

對我們的銀行轉帳，可以在開始任何操作之前，先取得 Terry 和 Bohr 帳戶 ID 的 lock，防止並發的轉帳互相干擾。

Distributed lock 可以用幾種技術實作，各有不同特性：

Redis with TTL：Redis 提供帶有自動過期的原子操作，非常適合 distributed lock。`SET` 指令加上 `NX`（只有不存在時才設定）和過期時間，可以原子性地建立一個 lock，Redis 會在 TTL 到期後自動移除它（`NX` flag 很關鍵，如果沒有它，第二個 process 可能覆寫掉已存在的 lock）。不需要額外的清理工作，Redis 在背景自動處理過期。Lock 是分散式的，因為所有 application server 都能存取同一個 Redis instance 並看到一致的狀態。優點是速度快、簡單；缺點是 Redis 成為單一故障點，需要處理 Redis 無法使用的情況。

Database column：用現有 database 加上 status 和 expiration 欄位來追蹤哪些資源被鎖住，從而實作 distributed lock。這個方式讓所有東西集中在一個地方，並利用 database 的 ACID 特性確保取得 lock 時的原子性。需要一個背景工作定期清理過期的 lock。優點是和現有資料一致、不需要額外基礎設施；缺點是 database 操作比 cache 操作慢，需要自己實作並維護清理邏輯。

ZooKeeper / etcd：這些是專門為分散式系統設計的協調 service，提供強一致性保證，即使在網路分割和 leader 故障的情況下也成立。ZooKeeper 使用 ephemeral node，當 client session 結束時自動消失，為崩潰的 process 提供自然的清理機制。兩者都使用 consensus 演算法（etcd 用 Raft，ZooKeeper 用 ZAB）來維護跨多個節點的一致性。優點是健壯，這些系統專門設計來處理 Redis 和 database 方案難以應對的複雜故障情境；缺點是運維複雜度，你需要另外運行和維護一個協調 cluster。

Distributed lock 不只是用於技術協調，它還能大幅改善用戶體驗，在 contention 發生之前就預防它。與其讓用戶競爭同一個資源，不如創造中間狀態，給予暫時的排他存取。

以 Ticketmaster 的座位預留為例。當你選擇一個座位，它不是立刻從「可用」變成「已售出」，而是進入「已預留」狀態，給你時間完成付款，同時阻止其他人選同一個座位。Contention 的窗口從整個購票流程（5 分鐘）縮小到了只有預留的那一步（毫秒）。

同樣的模式到處都是。Uber 把司機狀態設為 `pending_request`，電商平台把商品「暫時 hold 住」放進購物車，會議室預訂系統建立暫時的 hold。

buildmeat.org

優點是比複雜的 transaction 協調簡單；缺點是在高 contention 下 distributed lock 可能成為瓶頸，而且需要處理 lock timeout 和故障情境。

Saga Pattern

Saga pattern 採取了完全不同的方式。不像 2PC 那樣試圖把所有事情協調成原子的，它把操作拆成一系列獨立的步驟，每個步驟如果出錯都可以被「撤銷」（compensate）。

這樣想：不是同時鎖住 Terry 和 Bohr 的帳戶再協調，而是一步一步地執行操作。先從 Terry 的帳戶扣款並立刻 commit 那個 transaction；然後把 $100 存入 Bohr 的帳戶並立刻 commit。如果第二步失敗，就執行「補償操作」（compensation），把 $100 存回 Terry 的帳戶，撤銷第一步。

以我們的銀行轉帳為例：

- **第一步**：從 Database A 的 Terry 帳戶扣 $100，立刻 commit
- **第二步**：把 $100 存入 Database B 的 Bohr 帳戶，立刻 commit
- **第三步**：發送確認通知

如果第二步失敗（比如 Bohr 的帳戶不存在），就對第一步執行補償——把 $100 存回 Terry 的帳戶。如果第三步失敗，就補償第二步（從 Bohr 的帳戶扣款）和第一步（存回 Terry 的帳戶）。

每個步驟都是一個完整的、已 commit 的 transaction。沒有長時間開啟的 transaction，也不會有 coordinator 崩潰讓事情陷入不確定狀態。每個 database 操作獨立地成功或失敗。

但當然，有一個重要的取捨：**在 saga 執行過程中，系統暫時是不一致的。** 第一步完成後，Terry 的帳戶被扣款了，但 Bohr 的帳戶還沒入帳。其他 process 可能在這個窗口裡看到 Terry 的餘額少了 $100。如果有人去檢查系統裡的總金額，看起來暫時少了。

這種最終一致性（eventual consistency）是讓 saga 實際可行的原因。你用短暫的不一致性，換掉了 2PC 的脆弱性。處理方式是讓你的應用能理解這些中間狀態。比如，在所有步驟完成之前，把轉帳顯示為「處理中」。

如何選擇正確的做法

記住，跟很多系統設計問題一樣，沒有放諸四海皆準的答案。你需要根據具體的使用情境考量每種方式的取捨，並做出適當的判斷。

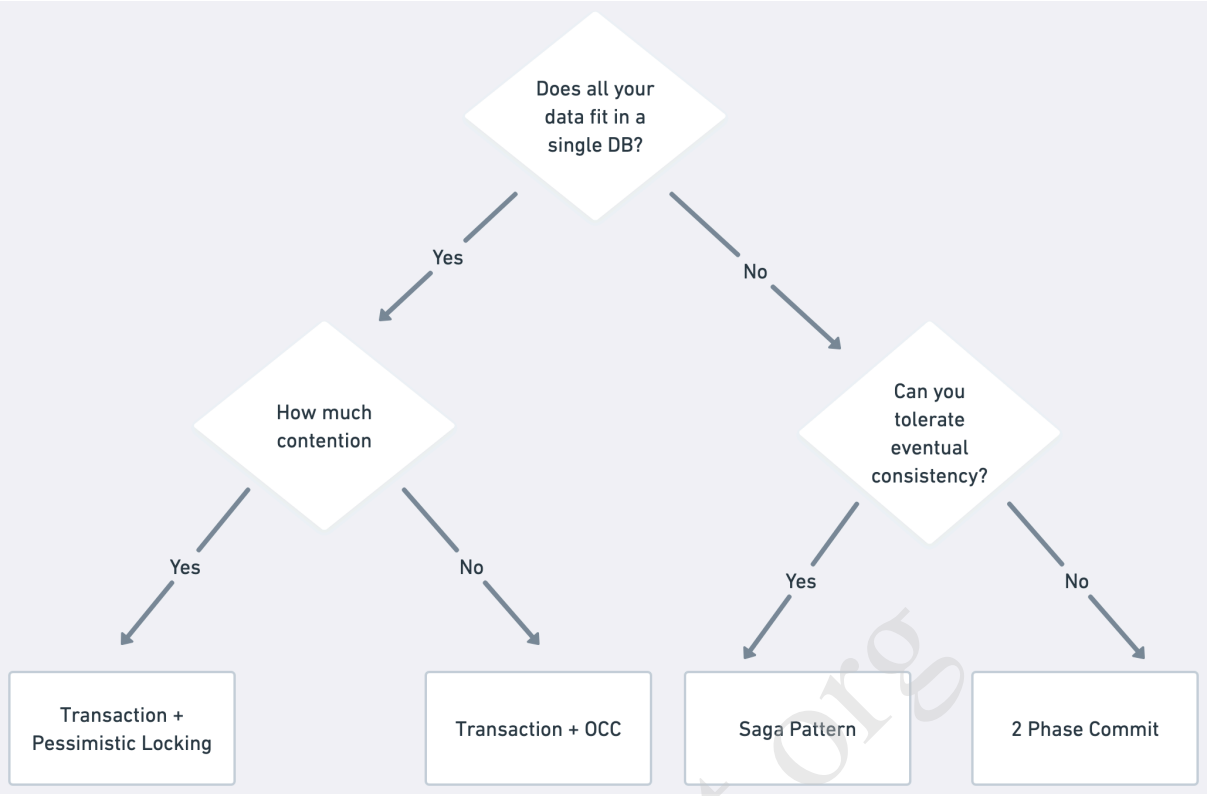
先從這個問題開始：你能把所有有 contention 的資料放在同一個 database 嗎？ 如果可以，就根據你的衝突頻率選擇 pessimistic locking 或 optimistic concurrency。

buildmost.org

- **單一 database，高 contention**：使用 pessimistic locking 加上明確 lock（`FOR UPDATE`）。效能可預測、好理解，在最壞的情況下也能妥善處理。
- **單一 database，低 contention**：使用 optimistic concurrency control，用現有的欄位作為版本。衝突少的時候效能更好，而且不會阻塞。
- **多個 database，必須保證原子性**：使用 distributed transaction（強一致性需求用 2PC，追求韌性則用 Saga）。只在絕對必要時才用。
- **用戶體驗很重要**：使用帶有預留機制的 distributed lock，避免用戶進入 contention 的情境。適合票務、電商，以及任何面向用戶的競爭流程。

方式	適合情況	不適合情況	典型延遲	複雜度
Pessimistic Locking	高 contention、嚴格一致性要求、單一 database	低 contention、高吞吐量需求	低（單一 DB query）	低
SERIALIZABLE Isolation	需要自動衝突偵測、無法事先確定要鎖哪些	效能關鍵、高 contention	中（衝突偵測 overhead）	低
Optimistic Concurrency	低 contention、讀多寫少、效能關鍵	高 contention、無法接受重試	低（無衝突時）	中
Distributed Transaction	必須跨系統保證原子性、可接受複雜度	高可用性需求、效能關鍵	高（網路協調）	非常高
Distributed Lock	面向用戶的流程、需要預留機制、比 2PC 簡單	純技術協調	低（簡單狀態更新）	中

buildnoat.org



如果不確定，就從 pessimistic locking 加上單一 database 開始。 簡單、可預測，之後永遠有機會改進。

什麼時候在面試裡用這些

不要等面試官來問 contention。主動識別多個 process 可能競爭同一個資源的情境，並提出適當的協調機制。這通常是你在梳理非功能性需求時，確定系統需要強一致性保證的時候。

識別信號

以下是幾個明確需要用到 contention 處理模式的情境：

- 多個用戶競爭有限資源：演唱會門票、拍賣競標、閃購庫存、媒合司機與乘客
- 防止重複預訂或重複扣款：付款處理、座位預留、會議室預訂
- 高並發下確保資料一致性：帳戶餘額更新、庫存管理、協同編輯
- 處理分散式系統中的 race condition：任何同一操作可能同時在多台 server 上發生、且結果對操作順序敏感的情境

常見面試情境

Contention 在面試題目裡出現的頻率非常高，是面試官最喜歡問的模式之一。

buildmoat.org

線上拍賣系統：非常適合展示 optimistic concurrency control，因為多個競標者在競爭同一個商品。可以用當前最高出價作為「版本號」，只接受高於期望當前出價的新出價。應用層的狀態協調也有幫助，比如把商品標記為「30 秒後結標」，防止最後一秒的 contention 情境。

活動訂票：雖然看起來像是座位選擇用 pessimistic locking 的經典場景，但應用層的狀態協調（座位預留）其實帶來更大的效益。當用戶選擇座位時，立刻把它預留起來並設定 10 分鐘的過期時間，這樣可以防止用戶填完付款資訊才發現座位被別人搶走的糟糕體驗。

銀行 / 支付系統：很好的地方展示 distributed transaction，因為不同銀行或 service 之間的帳戶轉帳需要跨系統的原子操作。應該先從韌性更好的 Saga pattern 開始，只有在面試官特別要求嚴格一致性時才提 2PC。

共乘派遣（Ride Sharing Dispatch）：應用層狀態協調在這裡大放異彩，派送叫車請求時把司機狀態設為 `pending_request`，防止對同一個司機同時發出多個請求。可以用帶 TTL 的 cache（司機 10 秒內沒回應就自動清除），或帶定期清理工作的 database status 欄位。

閃購 / 庫存系統：很適合展示多種方式的混合使用。庫存更新可以用 optimistic concurrency，以當前庫存數量作為版本；同時也要在購物車實作應用層的「hold」機制來改善用戶體驗，並減少結帳時的 contention。

Yelp / 評論系統：很好的 optimistic concurrency control 例子，因為用戶送出評論時需要更新餐廳的平均評分，同一家餐廳多個並發評論就製造了 contention。可以把當前評分和評論數當作「版本」，只有符合讀取時的值才更新，防止同時進來的評論讓評分計算出錯。

最好的候選人會在被問之前就主動識別 contention 問題：

> 「這個拍賣系統會有多個競標者同時競爭商品，所以我會使用 optimistic concurrency control，以當前最高出價作為版本檢查。」

> 「對票務系統，我想避免用戶填完付款資訊才發現座位被搶走，所以我會實作帶有 10 分鐘 timeout 的座位預留機制。」

> 「因為我們把用戶帳戶 shard 到不同的 database，不同 shard 之間的轉帳需要 distributed transaction。我會用 Saga pattern 來確保韌性。」

什麼時候不要過度設計

在不需要複雜機制的時候，不要硬用它。

一個很常見的錯誤，是候選人在一個簡單的 database transaction 加上 row lock 或 OCC 就夠的情況下，硬要用 distributed lock（Redis 等）。記住，加入新的元件就是

buildnoat.org

增加系統複雜度和引入新的故障模式，盡量避免。

- **低 contention 情境**：衝突很少發生的場景（例如只有管理員才能編輯的商品描述），用基本的 optimistic concurrency 加上重試邏輯就夠了，不需要設計複雜的 locking 機制。
- **單一用戶的操作**：個人 todo list、私人文件、用戶個人偏好設定，根本沒有 contention，不需要任何協調。
- **讀多寫少的工作負載**：大多數操作是讀取、偶爾才有寫入的情況，用簡單的 optimistic concurrency 處理偶發的寫入衝突就好，不需要影響讀取效能。

常見的 Deep Dive 問題

以下是面試中討論 contention 模式時最常見的追問。

「Pessimistic locking 怎麼防止 deadlock？」

想像兩個帳戶之間的銀行轉帳。Terry 要轉 $100 給 Bohr，同時 Bohr 要轉 $50 給 Terry。Transaction A 需要先鎖 Terry 的帳戶、再鎖 Bohr 的帳戶；Transaction B 需要先鎖 Bohr 的帳戶、再鎖 Terry 的帳戶。Transaction A 鎖住 Terry 的帳戶後，試著鎖 Bohr 的帳戶；Transaction B 鎖住 Bohr 的帳戶後，試著鎖 Terry 的帳戶。兩個 transaction 永遠互等對方釋放 lock，而這就是 deadlock。

Deadlock 發生的原因是兩個 transaction 以不同的順序取得 lock。業務邏輯本身不在乎順序，它只想在兩個用戶互動時更新兩者。但 database 沒辦法自動推斷哪些 lock 可以安全地同時取得。

標準解法是有序加鎖（ordered locking）：永遠按照一個一致的順序取得 lock，不管你的業務邏輯流程是什麼。按照某個確定性的 key（比如 user ID、database primary key，甚至記憶體地址）對你需要鎖住的資源排序。如果需要鎖住 user 123 和 user 456，永遠先鎖 123，即使你的業務邏輯先處理 456。這防止了循環等待，因為所有 transaction 都遵循同樣的取得順序。

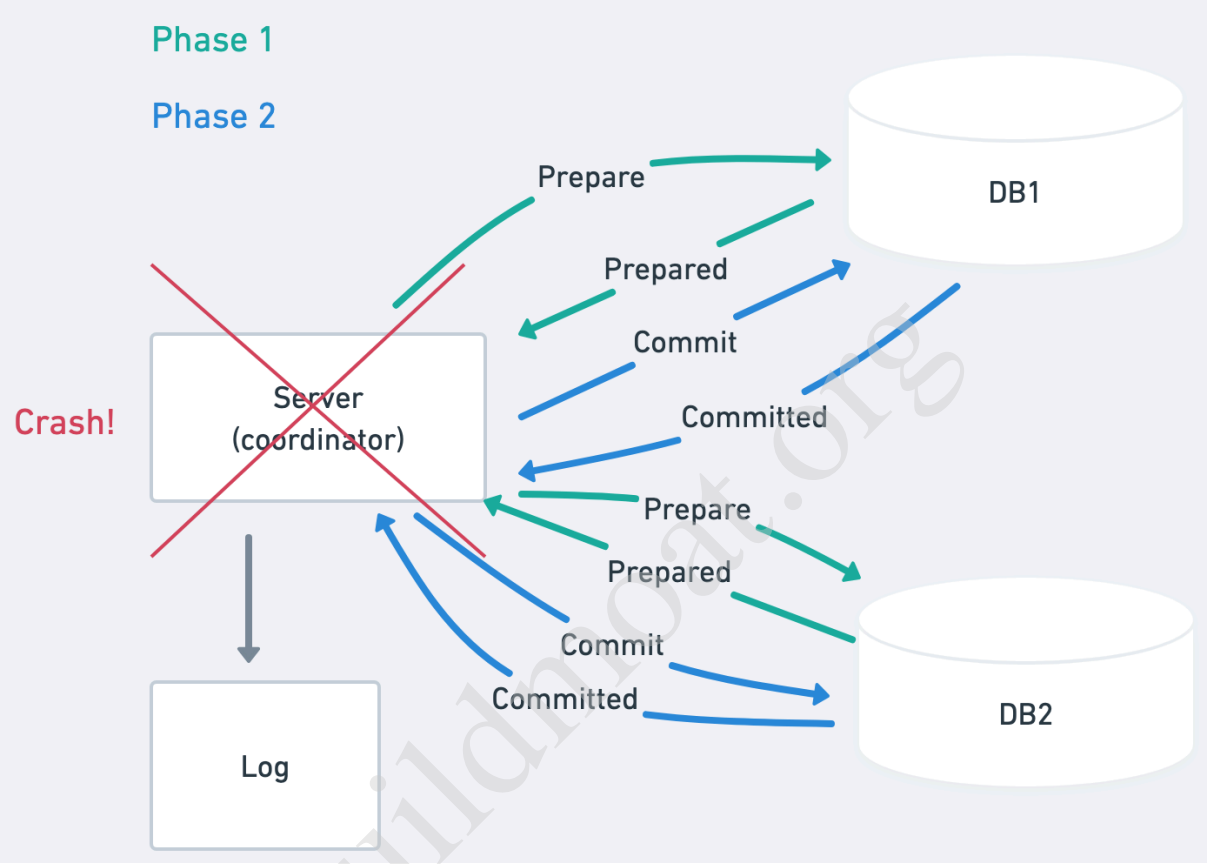
實際操作上，這意味著在取得任何 lock 之前，先把所有需要鎖的資源按照確定性的 key（例如 user ID）排序。對 user 456 和 user 123 之間的轉帳，永遠先鎖 user 123，不管是誰發起的轉帳。確切的排序方案不重要，只要它在系統中所有 transaction 之間保持全局一致就好。

作為後備方案，**database timeout 配置**是你的安全網，在有序加鎖不實用或有漏網之魚的時候使用。設定 transaction timeout，讓 deadlock 中的 transaction 在等待一段合理時間後被殺掉，然後重試。大多數現代 database 也有自動 deadlock 偵測，在偵測到循環時殺掉其中一個 transaction，但這應該是你的後備策略，不是主要策略。

bilunpai.org

「Distributed transaction 進行中 coordinator service 崩潰了怎麼辦？」

這是 2PC 的經典故障情境。Database 帶著已 prepare 的 transaction 在等待，但 commit 或 abort 的指令再也不會來了。那些 transaction 鎖住了資源，可能讓其他操作無限期地阻塞。



生產系統靠 **coordinator failover 和 transaction recovery** 來處理這個問題。新的 coordinator 啟動時，讀取持久化的 log，確認哪些 transaction 在進行中，然後把它們完成。大多數企業級 transaction manager 會自動處理這件事，但你仍然需要設計 coordinator 的高可用性，並在故障之間維護 transaction 狀態。

Saga 在這方面更有韌性（如前面所討論的），因為它不跨網路呼叫持有 lock。Coordinator 故障只是讓進度暫停，而不是讓 participant 陷入不確定狀態。

「OCC 的 ABA 問題怎麼處理？」

這是個狡猾的問題，測試你對更深層細節的理解。ABA 問題發生在這種情況：一個值在你讀取和寫入之間，從 A 變成 B 又變回 A。你的 optimistic check 看到相同的值，以為什麼都沒變，但其實已經發生了重要的狀態轉換。

buildmoat.org

以 Yelp 這樣的評論系統為例，用戶可以為商家評分，每個商家追蹤平均評分，這樣每次就不需要重算。一家餐廳有 4.0 星、100 則評論。同時進來兩則新評論，一個給 5 星，一個給 3 星。兩則評論都讀到當前平均是 4.0，然後各自計算新的平均。因為數學關係，最終平均可能還是落在 4.0 星，但現在有 102 則評論了。如果你只用平均評分作為「版本號」，兩個更新都會成功（因為都看到同樣的 4.0），但你漏掉了一則評論。

解法是用一個你知道永遠會改變的欄位。 以 Yelp 的案例，用 review count 而不是平均評分作為 optimistic concurrency 的版本依據。每則新評論都讓 count 遞增，所以它是一個完美的單調遞增版本號。你的更新變成「把新平均設為 X，並把 review count 增為 101，但只有在當前 count 是 100 的情況下才執行」：

```
-- 用 review count 作為「版本號」，因為它永遠會增加
UPDATE restaurants
SET avg_rating = 4.1, review_count = review_count + 1
WHERE restaurant_id = 'pizza_palace'
  AND review_count = 100;  -- 期望的當前 count
```

如果找不到自然會改變的欄位，就退而求其次，加一個明確的 version 欄位，每次更新都遞增，不管業務資料本身有沒有實際改變。就算平均評分沒有變，版本號也會顯示有處理過這件事。

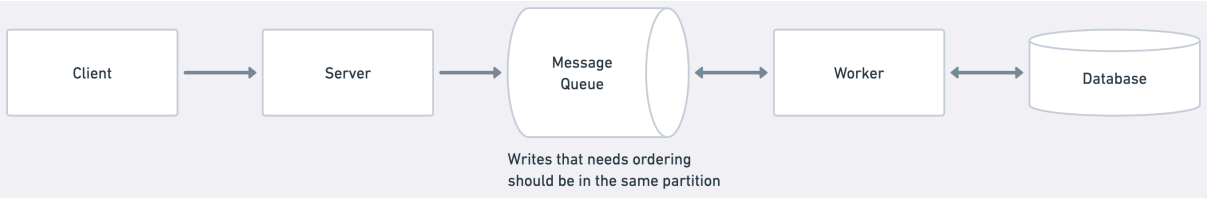
「當所有人都在搶同一個資源時，效能怎麼辦？」

這是 hot partition（熱分區）或 celebrity problem（名人問題），你精心設計的分散式系統，突然所有人都在打同一個點。想像一個名人加入 Twitter，幾百萬用戶試圖同時追蹤他們；或是一件稀有收藏品在 eBay 上架，幾千人同時競標同一件商品；或是 Taylor Swift 突然宣布一場演唱會，所有人同時搶票。

根本的問題是，當需求集中在單一點時，一般的擴展策略就失效了。Sharding 沒用，因為你沒辦法把一場 Taylor Swift 的演唱會拆成多個 database，大家要的就是那個特定的資源。Load balancing 沒用，因為 load balancer 只是把請求分散到不同的 server，但這些 server 最終都要競爭同一個 database row。就連 read replica 也沒用，因為瓶頸在寫入端。

你的第一個策略應該是質疑能不能改變問題本身，而不是把更多基礎設施砸進去。 也許與其只有一件拍賣品，其實你有 10 件一樣的東西，可以分開跑 10 場拍賣。也許對社群媒體互動而言，不需要立即一致性，用戶不會注意到追蹤操作要幾秒後才出現在名人的追蹤者數上。

waldmeat.org



對真的需要在熱資源上保持強一致性的情況，可以實作 **queue-based serialization（基於 Queue 的序列化）**。把所有對那個特定資源的請求放進一個專用的 queue，由單一一個 worker thread 來處理。這完全消除了 contention，因為操作變成循序的而不是並發的。Queue 作為緩衝，可以吸收流量峰值，而 worker 以可持續的速率處理請求。

取捨是延遲，用戶可能需要等更長的時間讓請求被處理。但這往往比讓整個系統在 contention 下停擺要好得多。

總結

Contention 處理對可靠系統至關重要，但成功的路徑並不是大多數工程師以為的那樣。**在考慮分散式協調之前，你應該把所有單一 database 的解法都用盡**，現代 database 如 PostgreSQL 可以處理幾十 TB 的資料和幾千個並發連接，這已經覆蓋了你會構建的絕大多數應用。而分散式協調的複雜度跨度巨大，帶來龐大的 overhead，效能往往還更差。

盡可能長時間地待在單一 database 裡，因為 pessimistic locking 和 optimistic concurrency 都能提供簡單、久經考驗的解法，並帶有 ACID 保證。Pessimistic locking 以可預測的方式處理高 contention，optimistic concurrency 在衝突少的時候提供絕佳的效能。只有當你真的需要跨系統保證原子性，或者需要地理分散部署時，才轉向分散式協調，而這種情況來得比大多數工程師預想的要晚得多。

最好的系統設計師，是那些努力把資料放在一起、並為他們具體的一致性需求選擇正確協調模式的人。把這些基礎掌握好，但永遠記住：能解決問題的最簡單方案，幾乎永遠是正確的選擇。

bulumoi.org